

Title: Customer Order Management Using Inheritance

■ Basic Information

Package: recordprog

Main Class: WeekForum

Concepts Used: Classes, Objects, Encapsulation, Constructors, Inheritance

■ Objective / Aim

To develop a Java program for managing customer orders using classes and inheritance, where SpecialOrder and NormalOrder inherit common properties and methods from the base Order class.

■ System Requirements

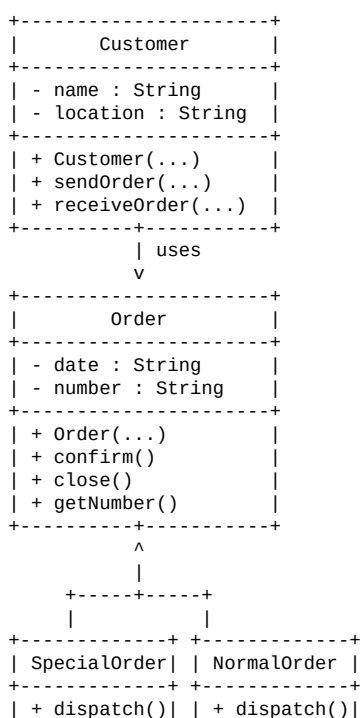
Hardware: Standard PC/Laptop, minimum 4 GB RAM, minimum 500 MB free storage.

Software: Windows/Linux/macOS, JDK, Java IDE such as Eclipse / IntelliJ IDEA / NetBeans / VS Code, Command Prompt / Terminal.

■ Algorithm / Logic Description

1. Start the program.
2. Define the Customer class with name and location and methods to send and receive orders.
3. Define the base Order class with date and order number.
4. Define confirm(), close(), and getNumber() methods in the Order class.
5. Define SpecialOrder as a subclass of Order with a dispatch() method.
6. Define NormalOrder as a subclass of Order with dispatch() and receive() methods.
7. Create a Customer object and SpecialOrder and NormalOrder objects.
8. Send, confirm, dispatch, receive where applicable, and close the orders.
9. Display the order-processing messages.
10. Stop the program.

■ UML Class Diagram



```

| | + receive() |
+-----+ +-----+

```

■ Program Flow

```

Start
|
v
Create Customer
|
v
Create SpecialOrder & NormalOrder
|
+--> Send Special Order
|
|   +--> Confirm --> Dispatch --> Close
|
+--> Send Normal Order
|
+--> Confirm --> Dispatch --> Receive --> Close
|
v
Customer receives order
|
v
End

```

■ Source Code

```
package recordprog;

//Customer class
class Customer {
    private String name;
    private String location;

    public Customer(String name, String location) {
        this.name = name;
        this.location = location;
    }

    public void sendOrder(Order order) {
        System.out.println(name + " from " + location + " is sending order " + order.getNumber());
    }

    public void receiveOrder(Order order) {
        System.out.println(name + " received order " + order.getNumber());
    }
}

//Base Order class
class Order {
    private String date;
    private String number;

    public Order(String date, String number) {
        this.date = date;
        this.number = number;
    }

    public void confirm() {
        System.out.println("Order " + number + " confirmed.");
    }

    public void close() {
        System.out.println("Order " + number + " closed.");
    }

    public String getNumber() {
        return number;
    }
}

//SpecialOrder inherits from Order
class SpecialOrder extends Order {
    public SpecialOrder(String date, String number) {
        super(date, number);
    }

    public void dispatch() {
        System.out.println("Special order " + getNumber() + " dispatched.");
    }
}

//NormalOrder inherits from Order
class NormalOrder extends Order {
    public NormalOrder(String date, String number) {
        super(date, number);
    }

    public void dispatch() {
        System.out.println("Normal order " + getNumber() + " dispatched.");
    }

    public void receive() {
        System.out.println("Normal order " + getNumber() + " received.");
    }
}

//Main class renamed as WeekFourum1
public class WeekForum {
    public static void main(String[] args) {
        Customer c1 = new Customer("Srikar", "Visakhapatnam");

        SpecialOrder so = new SpecialOrder("2026-08-10", "S0123");
        NormalOrder no = new NormalOrder("2026-08-10", "N0238");

        c1.sendOrder(so);
        so.confirm();
        so.dispatch();
        so.close();

        c1.sendOrder(no);
    }
}
```

```
        no.confirm();  
        no.dispatch();  
        no.receive();  
        no.close();  
  
        c1.receiveOrder(no);  
    }  
}
```

■ Compilation & Execution

Step 1: Save the file as **WeekForum.java** inside the appropriate package directory.

Step 2: Compile the program using:

```
javac -d . WeekForum.java
```

Step 3: Run the program using:

```
java recordprog.WeekForum
```

■■ Sample Output

```
Srikar from Visakhapatnam is sending order S0123
Order S0123 confirmed.
Special order S0123 dispatched.
Order S0123 closed.
Srikar from Visakhapatnam is sending order N0238
Order N0238 confirmed.
Normal order N0238 dispatched.
Normal order N0238 received.
Order N0238 closed.
Srikar received order N0238
```

■ Result

The Java program was successfully designed to demonstrate customer order processing using classes, encapsulation, constructors, and inheritance. SpecialOrder and NormalOrder extend the common Order class and perform their respective order operations.